

CĂUTARE BINARĂ

Clasa a IX-a | Informatică | Python

1. Ce este căutarea binară?

Căutarea binară este un algoritm de căutare a unui element într-un șir **SORTAT**. La fiecare pas, intervalul de căutare se înjumătățește, comparând elementul căutat cu cel din mijloc.

Condiție obligatorie: șirul trebuie să fie sortat crescător (sau descrescător, cu ajustări).

Comparație cu căutarea liniară:

- Căutare liniară: verifică pe rând fiecare element $\rightarrow O(n)$ pași
- Căutare binară: înjumătățește intervalul la fiecare pas $\rightarrow O(\log_2 n)$ pași
- Exemplu: pentru $n = 1.000.000$ elemente: liniară $\sim 1.000.000$ pași, binară ~ 20 pași

2. Algoritm: Căutare binară iterativă

Pașii algoritmului:

- Inițializează $st = 0$ (stânga), $dr = n-1$ (dreapta)
- Cât timp $st \leq dr$: calculează mijlocul $mij = (st + dr) // 2$
- Dacă $a[mij] == x$: am găsit elementul
- Dacă $a[mij] < x$: elementul e în dreapta, deci $st = mij + 1$
- Dacă $a[mij] > x$: elementul e în stânga, deci $dr = mij - 1$
- Dacă $st > dr$: elementul nu există în șir

Complexitate timp: $O(\log_2 n)$ – la fiecare pas intervalul se înjumătățește.

Spațiu: $O(1)$ – nu folosim memorie suplimentară.

Implementare Python:

```
def cautare_binara(a, x):  
    """Caută x în șirul sortat a."""  
    """Returnează indexul (0-based) sau -1 dacă nu există."""  
    st, dr = 0, len(a) - 1  
    while st <= dr:  
        mij = (st + dr) // 2  
        if a[mij] == x:  
            return mij                # am găsit  
        elif a[mij] < x:  
            st = mij + 1              # caută în dreapta  
        else:  
            dr = mij - 1              # caută în stânga  
    return -1                        # negăsit
```

```
# Testare
sir = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(cautare_binara(sir, 23))    # 5   (indexul 5)
print(cautare_binara(sir, 10))    # -1  (nu există)
```

3. Probleme rezolvate

Problema 1 - Căutare element în șir sortat

Se citește n , apoi n numere în ordine crescătoare, și un număr x . Afișați poziția lui x în șir (indexat de la 1), sau 0 dacă nu există.

Date intrare:	5 2 5 8 12 23 8
Date ieșire:	3

Indiciu: Aplică căutarea binară standard. Dacă găsești elementul la indexul i (0-based), afișează $i+1$.

Rezolvare:

```
n = int(input())
a = list(map(int, input().split()))
x = int(input())

st, dr, rez = 0, n - 1, 0
while st <= dr:
    mij = (st + dr) // 2
    if a[mij] == x:
        rez = mij + 1
        break
    elif a[mij] < x:
        st = mij + 1
    else:
        dr = mij - 1
print(rez)
```

Problema 2 - Prima poziție mai mare sau egală cu x

Se citește n , un șir sortat crescător de n elemente și un număr x . Afișați prima poziție i (1-indexed) cu $a[i] \geq x$, sau $n+1$ dacă nu există.

Date intrare:	6 1 3 5 7 9 11 6
Date ieșire:	4

Indiciu: Variantă de căutare binară care găsește limita stângă (lower bound). Reține poziția ori de câte ori condiția e satisfăcută și mută *dr* spre stânga.

Rezolvare:

```
n = int(input())
a = list(map(int, input().split()))
x = int(input())

st, dr, rez = 0, n - 1, n + 1
while st <= dr:
    mij = (st + dr) // 2
    if a[mij] >= x:
        rez = mij + 1
        dr = mij - 1
    else:
        st = mij + 1
print(rez)
```

Problema 3 - Numărarea aparițiilor unui element

Se citește *n*, un șir sortat crescător de *n* elemente și un număr *x*. Afișați de câte ori apare *x* în șir.

Date intrare:	7 1 3 3 3 5 7 9 3
Date ieșire:	3

Indiciu: Găsește prima și ultima poziție a lui *x* folosind două căutări binare (lower bound și upper bound). Răspuns = ultima - prima + 1.

Rezolvare:

```
n = int(input())
a = list(map(int, input().split()))
x = int(input())
```

```
def lower(a, x):          # prima pozitie cu a[i] == x
    st, dr, rez = 0, len(a)-1, -1
    while st <= dr:
        mij = (st + dr) // 2
        if a[mij] == x: rez = mij; dr = mij - 1
        elif a[mij] < x: st = mij + 1
        else: dr = mij - 1
    return rez

def upper(a, x):          # ultima pozitie cu a[i] == x
    st, dr, rez = 0, len(a)-1, -1
    while st <= dr:
        mij = (st + dr) // 2
        if a[mij] == x: rez = mij; st = mij + 1
        elif a[mij] < x: st = mij + 1
        else: dr = mij - 1
    return rez

p, u = lower(a, x), upper(a, x)
print(0 if p == -1 else u - p + 1)
```

Sursa probleme/teorie

Problemele și teoria din această fișă sunt inspirate din platforma pbinfo.ro, categoria 'Căutare binară'.